# In-Class Activity 07: Flutter SQLite (Local Storage Part 1)

📅 **Due Date: 03/02/2026**

🕑 **Time: TBA**

Welcome to the world of persistent data. Today, we will build a Flutter app that stores and retrieves data even after the app is closed.

## 🎯 Learning Objective

Create a simple Flutter app that integrates a local **SQLite database** using the `sqflite` package. This activity provides practical, hands-on experience with configuring databases, creating helper classes, and performing CRUD (Create, Read, Update, Delete) operations in a mobile environment.

## 📦 What to Submit

📄 **Main Dart File**
**(main.dart)**

🛠️ **Helper File**
**(database_helper.dart)**

⚙️ **YAML File**
**(pubspec.yaml)**

# 🚀 Step-by-Step Guide

## 1 Project Setup

Start by creating a brand new Flutter project. You can name it `flutter_database`.

> 💡 **Quick Tip:** Use the command `flutter create flutter_database` in your terminal or use the "New Project" wizard in VS Code/Android Studio.

### Add Dependencies

Open your `pubspec.yaml` file. We need to add three specific packages to handle database operations and file paths.

```yaml
dependencies:
  flutter:
    sdk: flutter

  # Add these lines:
  sqflite: ^2.2.2
  path_provider: ^2.0.11
  path: ^1.8.2
```

> ⚠️ **Important:** Indentation matters in YAML files! Ensure these lines are aligned with the `flutter` dependency. After saving, run `flutter pub get`.

## 2 Create the Database Helper

It's best practice to separate database logic from UI code. We will create a "Singleton" class to manage our database connection.

Create a new file in your `lib` folder named `database_helper.dart` and paste the following code:

▶ 📝 **Click to view complete database_helper.dart code**

## 3 Build the UI Logic

Now, let's connect our database helper to the screen. We'll replace `main.dart` with a layout containing buttons for Insert, Query, Update, and Delete.

> 🎨 **Design Challenge:** The code below is minimal. Add some `Padding`, `Borders`, or change the `ButtonStyles` to make your UI cleaner and more visually appealing!

▶ 📝 **Click to view complete main.dart code**

# 🛠️ Part 2 Requirements (Challenge)

Once you have the basic CRUD operations working, enhance your app with the following features:

- ✓ **Query by ID:** Create a function in `DatabaseHelper` to search for a specific record using its ID. Add a button to your UI to test this.
- ✓ **Delete All Records:** Create a function to clear the entire table. This is very useful for resetting your app state!

> 🧠 **Hint:** For "Delete All", you can use the SQL command `DELETE FROM my_table` without a `WHERE` clause.

# 🧪 Testing & Debugging

Since we don't have a visual list in the UI yet, we will rely on the **Debug Console**.

> ✅ **How to Verify:**
>
> 1. Run the app on an Emulator or Physical Device.
> 2. Open the **Run** tab in Android Studio or VS Code.
> 3. Tap the "Insert" button. Look for `inserted row id: 1` in the console.
> 4. Tap "Query". You should see the data map `{_id: 1, name: Bob, age: 23}` printed.
> 5. Try Update and Delete and watch the console confirm the actions.

# 🧠 Critical Thinking (What, Why, How)

Answer all questions below in a **Microsoft Word document (.doc or .docx)**. Write complete responses that explain your decisions and understanding of each task.

> **Submission Format:** Name your file as `LastName_FirstName_CriticalThinking_InClass07.docx` and submit it with your code files.

## What

1. What problem is solved by creating a separate `DatabaseHelper` file instead of placing SQL logic in `main.dart`?
2. What happens to user data when the app closes, and how does SQLite support persistence?
3. What is the role of each CRUD action in your app workflow?

## Why

4. Why is `await dbHelper.init()` required before calling insert/query/update/delete?
5. Why should update and delete operations target records by ID instead of by name or age?
6. Why might a local SQLite database be a better choice than a remote API for this activity?

## How

7. How would you verify that each button is working correctly using only Debug Console output?
8. How would you modify the app to show query results on screen instead of printing to console?
9. How could you prevent accidental data loss when implementing “Delete All Records”?

## 📊 Quick Grading Rubric

- **Setup & Dependencies (20%)**: Correct project setup and required packages added in `pubspec.yaml`.
- **Database Helper (25%)**: Functional `database_helper.dart` with table creation and core CRUD methods.
- **UI & CRUD Actions (25%)**: Buttons are wired correctly and actions run without errors.
- **Testing Evidence (15%)**: Clear Debug Console output showing insert, query, update, and delete behavior.
- **Critical Thinking Responses (15%)**: Answers are complete, specific, and explain what, why, and how.

## 🔄 Alternative Database Solutions

While `sqflite` is the standard, consider these popular alternatives for future projects:

- **Isar:** Extremely fast, NoSQL database.
- **Hive:** Lightweight key-value database, written in pure Dart.
- **Moor (Drift):** Reactive persistence library, built on top of SQLite.
- **Floor:** SQLite abstraction that provides an easy mapping between objects and database rows.

---

**Resources:** SQFlite Documentation | Flutter SQLite Example

Happy Coding! 💻 ✨